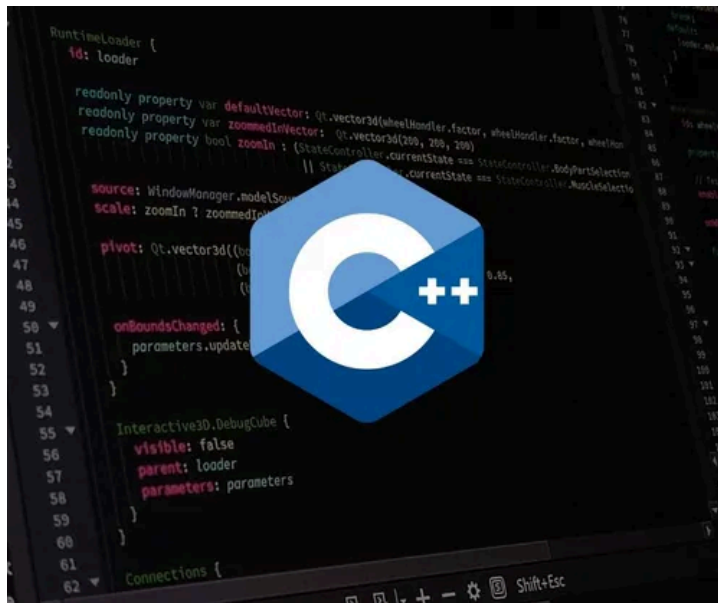


Materia: Laboratorio de Programación
EEST N.º 1 - “Eduardo Ader” Vicente López
5º año 3º Grupos A-B miércoles y jueves de 15:15 a 17:15 Hs



Actividad 6 — Taller Práctico de Repaso y Consolidación en C++

Estudiante: Lucas Benitez

Curso: 5to 3ra

Docentes: Yamil Ganduglia y Mansilla Muñoz York Elías

Repositorio: <https://github.com/lukitanahuelbenitez-tech/repasogeneral>

Control de revisiones

Versión	Fecha	Autor	Descripción
v1.0.0 Baseline Aprobado	08/10/2026	Lucas Benitez	Versión inicial del informe de la Actividad 6. Incluye las respuestas al cuestionario técnico.

Cuestionario técnico de control

1. Caso base

Si una función recursiva no tiene un caso base que detenga las llamadas, puede seguir llamándose a sí misma indefinidamente. Esto provoca que se llene la pila de memoria (*stack*) y se produzca un error que puede interrumpir la ejecución del programa.

2. Eficiencia

La instrucción **break** permite detener un ciclo cuando se encuentra el elemento buscado. De esta manera, el programa no sigue recorriendo los elementos que quedan en el arreglo, evitando operaciones innecesarias y mejorando el rendimiento de la búsqueda.

3. Direccionamiento

Podemos utilizar el operador **&** para mostrar la dirección de memoria de cada elemento del arreglo. Al comparar las direcciones de elementos consecutivos, podemos observar que están separados por una distancia equivalente al tamaño de cada elemento.

4. Variables temporales

La variable temporal permite guardar uno de los valores antes de intercambiarlos, evitando que se pierda. Primero se guarda el valor de la primera variable, después se copia el valor de la segunda en la primera y, finalmente, se asigna el valor guardado a la segunda. En el programa se utiliza la variable `temporal` dentro de la función `intercambiarValores()`.

5. Autoevaluación

El reto que más me costó comprender fue el intercambio de valores mediante punteros, porque tuve que entender cómo se utilizan las direcciones de memoria para modificar las variables originales. Lo pude resolver revisando el funcionamiento de los operadores `&` y `*` y siguiendo paso a paso cómo la variable temporal permite intercambiar los valores sin perder ninguno.

Figuras

```
PS C:\Users\lukit\OneDrive\Desktop\Actividades LPR 5to\repasogeneral> git remote -v
origin  https://github.com/lukitanahuelbenitez-tech/repasogeneral.git (fetch)
origin  https://github.com/lukitanahuelbenitez-tech/repasogeneral.git (push)
origin  https://gitlab.com/lukitanahuelbenitez-tech/repasogeneral.git (push)
```

Figura 1: Configuración de Git con doble destino de subida (push) hacia los repositorios remotos de GitHub y GitLab mediante el comando `git remote -v`.

Materia: Laboratorio de Programación
EEST N.º 1 - "Eduardo Ader" Vicente López
5º año 3º Grupos A-B miércoles y jueves de 15:15 a 17:15 Hs

```
=====
TALLER INTEGRADOR REPASO - ESTUDIANTE: LUCAS BENITEZ
=====

=== RETO 1: SUMA RECURSIVA ===
Suma acumulada de 1 hasta 5: 15

=== RETO 2: BÚSQUEDA EN MEMORIA CONTIGUA ===
[SISTEMA] Valor 18 hallado en el índice [6]
Direccion RAM Hexadecimal: 0x7d255ff868

=== RETO 3: INTERCAMBIO DE CELDAS ===
Valores previos -> X: 100 | Y: 500
Valores actuales -> X: 500 | Y: 100
=====
```

Figura 2: Ejecución del programa en C + + y resultados de los tres retos: suma recursiva, búsqueda secuencial en un arreglo e intercambio de valores mediante punteros.